

Part 1: Linear Regression on the Tips Dataset

To begin, let's load the tips dataset from the `seaborn` library. This dataset contains records of tips, total bill, and information about the person who paid the bill. **We'll be trying to predict tips from the other data.**

Data Loading and Feature Engineering

```
In [ ]: import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.feature_extraction import DictVectorizer
import matplotlib.pyplot as plt
np.random.seed(42)
plt.style.use('fivethirtyeight')
sns.set()
sns.set_context("talk")
%matplotlib inline
```

```
In [ ]: data = sns.load_dataset("tips")

print("Number of Records:", len(data))
data.head()
```

Number of Records: 244

```
Out[ ]:   total_bill  tip  sex  smoker  day  time  size
0      16.99  1.01 Female     No  Sun  Dinner     2
1      10.34  1.66  Male     No  Sun  Dinner     3
2      21.01  3.50  Male     No  Sun  Dinner     3
3      23.68  3.31  Male     No  Sun  Dinner     2
4      24.59  3.61 Female     No  Sun  Dinner     4
```

```
In [ ]: y = data['tip']
X = data.drop(columns='tip')
```

We do some feature engineering to convert the categorical columns into one-hot coding.

```
In [ ]: def one_hot_encode_revised(data):
    """
    Return the one-hot encoded dataframe of our input data, removing redun-
```

Parameters

data: a dataframe that may include non-numerical features

Returns

A one-hot encoded dataframe that only contains numeric features without

"""

```
columns = ['sex', 'smoker', 'day', 'time']
```

```
for column in columns:
```

```
    values = data[column].unique()
```

```
    for value in values[:-1]:
```

```
        data[column + '=' + value] = (data[column] == value).astype(int)
```

```
    data = data.drop(column, axis=1)
```

```
return data
```

```
In [ ]: X = one_hot_encode_revised(X)
```

```
In [ ]: X.head()
```

```
Out [ ]:
```

	total_bill	size	sex=Female	smoker=No	day=Sun	day=Sat	day=Thur	time=Di
--	------------	------	------------	-----------	---------	---------	----------	---------

0	16.99	2	1	1	1	0	0
1	10.34	3	0	1	1	0	0
2	21.01	3	0	1	1	0	0
3	23.68	2	0	1	1	0	0
4	24.59	4	1	1	1	0	0

Insert code below to perform a train/test split on X and Y, with 80/20 ratio

```
In [ ]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

Linear regression with the model from Sklearn

We start with using the linear regression model from sklearn library

Insert code below to create a linear regression model, train the model on the training set

```
In [ ]: from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
Out[ ]: LinearRegression
LinearRegression()
```

Insert code below to make a prediction on the test dataset, and calculate the mean square error

```
In [ ]: from sklearn.metrics import mean_squared_error

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse:.3f}")
```

Mean Squared Error: 0.703

The mean square error should be around 0.7 if your implementation is correct.

Linear regression with self-defined model

```
In [ ]: X.head()
```

```
Out[ ]:
```

	total_bill	size	sex=Female	smoker=No	day=Sun	day=Sat	day=Thur	time=Di
0	16.99	2	1	1	1	0	0	
1	10.34	3	0	1	1	0	0	
2	21.01	3	0	1	1	0	0	
3	23.68	2	0	1	1	0	0	
4	24.59	4	1	1	1	0	0	

Now that all of our data is numeric, we can begin to define our model function. Notice that after one-hot encoding our data, we now have 8 features instead of 6. Therefore, our linear model now looks like:

$$\text{Tip} = \theta_1 \cdot \text{total_bill} + \theta_2 \cdot \text{billsize} + \dots$$

We can represent the linear combination above as a matrix-vector product.

Below, we create a `MyLinearModel` class with two methods, `predict` and `fit`.

The `predict` method gives a prediction based on the linear model:

$$\text{prediction} = \theta \cdot X^T$$

, where $\theta = [\theta_1, \theta_2, \dots]$, $X = [X_1, X_2, \dots]$.

The `fit` method should train the model (i.e., calculating the best parameters θ that minimizes the error). But for now we just set all of its parameters to zero.

```
In [ ]: class MyLinearModel():
        def predict(self, X):
            return X @ self._thetas

        def fit(self, X, y):
            number_of_features = X.shape[1]
            self._thetas = np.zeros(shape = (number_of_features, 1))
```

Here is an example to creating a linear model, fit the model. Since we only set zeros to theta in the `fit` method, `model._thetas` gives all zeros.

```
In [ ]: model = MyLinearModel()
        model.fit(X_train, y_train)

        print("model parameters:\n", model._thetas)
```

```
model parameters:
[[0.]
 [0.]
 [0.]
 [0.]
 [0.]
 [0.]
 [0.]
 [0.]]
```

Insert code below, check the mean square error of this model on both training set, and test set

```
In [ ]: from sklearn.metrics import mean_squared_error

        y_train_pred_zeros = model.predict(X_train)
        mse_train_zeros = mean_squared_error(y_train, y_train_pred_zeros)
        print(f"Mean Squared Error on training set (all zeros): {mse_train_zeros:.3f}")

        y_test_pred_zeros = model.predict(X_test)
        mse_test_zeros = mean_squared_error(y_test, y_test_pred_zeros)
        print(f"Mean Squared Error on test set (all zeros): {mse_test_zeros:.3f}")
```

```
Mean Squared Error on training set (all zeros): 11.566
Mean Squared Error on test set (all zeros): 8.230
```

Fitting a Linear Model using `scipy.optimize.minimize` Methods

We will use the `scipy.optimize.minimize` function, to implement the fit method of the linear model.

Note that we've added a `loss_function` parameter where the model is fit using the desired loss function.

```
In [ ]: from scipy.optimize import minimize

def l1(y, y_hat):
    return np.abs(y - y_hat)

def l2(y, y_hat):
    return (y - y_hat)**2

class MyLinearModel():
    def predict(self, X):
        return X @ self._thetas

    def fit(self, loss_function, X, y):
        """
        Produce the estimated optimal _thetas for the given loss function,
        feature matrix X, and observations y.

        Parameters
        -----
        loss_function: either the squared or absolute loss functions defined
        X: a 2D dataframe (or numpy array) of numeric features (one-hot enco
        y: a 1D vector of tip amounts

        Returns
        -----
        The estimate for the optimal theta vector that minimizes our loss
        """

        # The code starts with an initial guess (starting_guess) of all zero
        # It then computes the average loss using the given loss_function (e
        # The minimize function adjusts the theta values to reduce the loss.
        # This process is akin to gradient descent but handled internally by
        # which uses more advanced optimization techniques (e.g., BFGS) to f
        number_of_features = X.shape[1]

        starting_guess = np.zeros(number_of_features)
        self._thetas = minimize(lambda theta:
                                np.mean(loss_function(y, X@theta))
                                , x0 = starting_guess)['x']

        # Notice above that we extract the 'x' entry in the dictionary retur
        # This entry corresponds to the optimal theta estimated by the funct
        # we know it's a little confusing, but 'x' is hard coded into the mi
        # because of the fact that in the optimization universe "x" is what
        # It'd be less confusing if they used "theta".
```

Now, we create a new linear model, fit the model with `l2` loss, and show the trained parameters θ .

```
In [ ]: # When you run the code below, you should get back some non zero thetas.

model = MyLinearModel()
model.fit(l2, X_train, y_train)
model._thetas
```

```
Out[ ]: array([ 0.09637801,  0.2776863 ,  0.02506293,  0.20111445, -0.03052356,
                -0.15324107,  0.30006814,  0.36144962])
```

Insert code below, print the mean square error of this model on both training set, and test set

```
In [ ]: y_train_pred_l2 = model.predict(X_train)
mse_train_l2 = mean_squared_error(y_train, y_train_pred_l2)
print(f"Mean Squared Error on training set (L2 loss): {mse_train_l2:.3f}")

y_test_pred_l2 = model.predict(X_test)
mse_test_l2 = mean_squared_error(y_test, y_test_pred_l2)
print(f"Mean Squared Error on test set (L2 loss): {mse_test_l2:.3f}")
```

Mean Squared Error on training set (L2 loss): 1.114

Mean Squared Error on test set (L2 loss): 0.728

Implementing `fit` Function with Gradient Descent

To modify the `fit` function to use gradient descent instead of `scipy.optimize.minimize`, we need to update the theta parameters iteratively by computing the gradient of the loss function and updating the parameters in small steps (i.e., a small learning rate).

```
In [ ]: import numpy as np

class MyLinearModel():
    def predict(self, X):
        return X @ self._thetas

    def fit(self, loss_function, X, y, learning_rate=0.001, iterations=100):
        """
        Fit the model using gradient descent to minimize the loss function.

        Parameters
        -----
        loss_function: either the squared loss (l2) or absolute loss (l1) fu
        X: a 2D numpy array of numeric features.
        y: a 1D numpy array of the target values (regression targets).
        learning_rate: step size for gradient descent.
```

```

iterations: number of gradient descent iterations to run.

Returns
-----
Optimal theta values after performing gradient descent.
"""

# Initialize theta parameters to zeros
number_of_features = X.shape[1]
self._thetas = np.zeros(number_of_features)

# Iterate to perform gradient descent
m = len(y) # Number of training examples

for i in range(iterations):
    predictions = self.predict(X) # Compute predictions: X @ theta

    # Calculate the gradient of the loss
    if loss_function == l2:
        # For squared loss (l2)
        errors = predictions - y
        gradient = (2/m) * X.T @ errors
    elif loss_function == l1:
        # For absolute loss (l1)
        errors = np.sign(predictions - y)
        gradient = (1/m) * X.T @ errors

    # Update the parameters using the gradient
    self._thetas -= learning_rate * gradient

return self._thetas

```

In []: *# When you run the code below, you should get back some non zero thetas.*

```

model = MyLinearModel()
model.fit(l2, X_train, y_train, learning_rate=0.0005, iterations=50)
model._thetas

```

Out[]: 0

total_bill	0.141020
size	0.033002
sex=Female	0.005123
smoker=No	0.011583
day=Sun	0.005112
day=Sat	0.001667
day=Thur	0.004558
time=Dinner	0.007314

dtype: float64

Insert code cell below to calculate the mean square error of the model on test set

```
In [ ]: y_test_pred_gd = model.predict(X_test)
mse_test_gd = mean_squared_error(y_test, y_test_pred_gd)
print(f"Mean Squared Error on test set (Gradient Descent L2 loss): {mse_test
```

Mean Squared Error on test set (Gradient Descent L2 loss): 0.723

Insert code cell below, train a linear model with l1 loss, print out the mean square error on test set

```
In [ ]: model_l1 = MyLinearModel()
model_l1.fit(l1, X_train, y_train, learning_rate=0.0005, iterations=50)

y_test_pred_l1 = model_l1.predict(X_test)
mse_test_l1 = mean_squared_error(y_test, y_test_pred_l1)
print(f"Mean Squared Error on test set (Gradient Descent L1 loss): {mse_test
```

Mean Squared Error on test set (Gradient Descent L1 loss): 0.764

Part 2: Gradient Descent on Sinusoidal Data

Load the csv "lab8_data" from Google Drive

```
In [ ]: # Load the Drive helper and mount
from google.colab import drive

# Prompt for Authorization
drive.mount('/content/drive')
```

Mounted at /content/drive


```
In [ ]: !ls /content/drive/MyDrive/ECEN250/Lab8/lab8_data.csv
```

```
/content/drive/MyDrive/ECEN250/Lab8/lab8_data.csv
```

```
In [ ]: # Run this cell to load the data for this problem
df = pd.read_csv("/content/drive/MyDrive/ECEN250/Lab8/lab8_data.csv", index_
df.head()
```

```
Out[ ]:
```

	x	y
0	-5.000000	-7.672309
1	-4.966555	-7.779735
2	-4.933110	-7.995938
3	-4.899666	-8.197059
4	-4.866221	-8.183883

This data has only two columns - `x` and `y`.

If we plot this data, we see that there is a clear sinusoidal relationship between `x` and `y`.

```
In [ ]: import plotly.express as px
px.scatter(df, x = "x", y = "y")
```

In this exercise, we'll show gradient descent is so powerful it can even optimize a nonlinear model. Specifically, we're going to model the relationship of our data by:

$$f_{\theta(\cdot)} = \theta_1 x + \sin(\theta_2 x)$$

Our model is parameterized by both θ_1 and θ_2 , which we can represent in the vector, θ .

Note that a general sine function $a \sin(bx + c)$ has three parameters: amplitude scaling parameter a , frequency parameter b and phase shifting parameter c .

Here, we're assuming the amplitude a is around 1, and the phase shifting parameter c is around zero. We do not attempt to justify this assumption and you're welcome to see what happens if you ignore this assumption.

You might ask why we don't just create a linear model like we did earlier with a sinusoidal feature. The issue is that the θ is **INSIDE** the sin function. In other words, linear models use their parameters to adjust the scale of each feature, but θ_2 in this model adjusts the frequency of the feature. There are tricks we could play to use our linear model framework here, but we won't attempt this in our lab.

We define the `sin_model` function below that predicts $\mathbf{y}$ (the y -values) using $\mathbf{x}$ (the x -values) based on our new equation.

```
In [ ]: def sin_model(x, theta):
        """
        Predict the estimate of y given x, theta_1, theta_2

        Keyword arguments:
        x -- the vector of values x
        theta -- a vector of length 2, where theta[0] = theta_1 and theta[1] = theta_2
        """
        theta_1 = theta[0]
        theta_2 = theta[1]
        return theta_1 * x + np.sin(theta_2 * x)
```

Recall $\hat{\theta}$ is the value of θ that minimizes our loss function. One way of solving for $\hat{\theta}$ is by computing the gradient of our loss function with respect to θ . Recall that the gradient is a column vector of two partial derivatives.

Below are the expressions for calculating the gradients. Use them to fill in the functions below.

$L(\mathbf{x}, \mathbf{y}, \theta_1, \theta_2)$: our loss function, the mean squared error

$$L(\mathbf{x}, \mathbf{y}, \theta_1, \theta_2) = \frac{1}{n} \sum_{i=1}^n (\sin_model(x_i, \theta_1, \theta_2) - y_i)^2 = \frac{1}{n} \sum_{i=1}^n (\theta_1 x_i + \sin(\theta_2 x_i) - y_i)^2$$

$\frac{\partial L}{\partial \theta_1}$: the partial derivative of L with respect to θ_1

$$\frac{\partial L}{\partial \theta_1} = \frac{1}{n} \sum_{i=1}^n 2 \cdot (\theta_1 x_i + \sin(\theta_2 x_i) - y_i) \cdot x_i$$

$\frac{\partial L}{\partial \theta_2}$: the partial derivative of L with respect to θ_2 ,

$$\frac{\partial L}{\partial \theta_2} = \frac{1}{n} \sum_{i=1}^n 2 \cdot (\theta_1 x_i + \sin(\theta_2 x_i) - y_i) \cdot \cos(\theta_2 x_i) \cdot x_i$$

Recall that $L(\mathbf{x}, \mathbf{y}, \theta_1, \theta_2) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Complete the functions in the code cell below

Specifically, the functions `sin_MSE`, `sin_MSE_dt1` and `sin_MSE_dt2` should compute L , $\frac{\partial L}{\partial \theta_1}$ and $\frac{\partial L}{\partial \theta_2}$ respectively. Use the expressions you wrote for $\frac{\partial L}{\partial \theta_1}$ and $\frac{\partial L}{\partial \theta_2}$ to implement these functions. In the functions below, the parameter `theta` is a vector that looks like $\begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$. We have completed `sin_MSE_gradient`, which calls `dt1` and `dt2` and returns the gradient `dt` for you.

Notes:

- Keep in mind that we are still working with our original set of data, `df`.
- To keep your code a bit more concise, be aware that `np.mean` does the same thing as `np.sum` divided by the length of the numpy array.
- Another way to keep your code more concise is to use the function `sin_model` we defined which computes the output of the model.

```
In [ ]: # This function is already implemented for you.
def sin_MSE(theta, x, y):
    """
    Compute the numerical value of the l2 loss of our sinusoidal model given

    Keyword arguments:
    theta -- the vector of values theta
    x      -- the vector of x values
    y      -- the vector of y values
    """
    return np.mean((sin_model(x, theta) - y)**2)

# Complete this function.
def sin_MSE_dt1(theta, x, y):
    """
    Compute the numerical value of the partial of l2 loss with respect to th

    Keyword arguments:
    theta -- the vector of values theta
    x      -- the vector of x values
    y      -- the vector of y values
    """
    # add your code here
    error = sin_model(x, theta) - y
    return np.mean(2 * error * x)

# Complete this function.
def sin_MSE_dt2(theta, x, y):
    """
```

```

    Compute the numerical value of the partial of l2 loss with respect to th

    Keyword arguments:
    theta -- the vector of values theta
    x      -- the vector of x values
    y      -- the vector of y values
    """
    # add your code here
    error = sin_model(x, theta) - y
    return np.mean(2 * error * np.cos(theta[1] * x) * x)

# This function calls dt1 and dt2 and returns the gradient dt. It is already
def sin_MSE_gradient(theta, x, y):
    """
    Returns the gradient of l2 loss with respect to vector theta

    Keyword arguments:
    theta -- the vector of values theta
    x      -- the vector of x values
    y      -- the vector of y values
    """
    return np.array([sin_MSE_dt1(theta, x, y), sin_MSE_dt2(theta, x, y)])

```

Use the following code to verify if your implementation is correct.

```

In [ ]: # Use synthetic data to verify the correctness of sin_MSE_dt1(theta, x, y)
        theta = np.array([1, 1])
        x = np.array([1, 2, 3])
        y = np.array([1, 2, 3])

        # A correct implementation should give a result around 2.056
        print(sin_MSE_dt1(theta, x, y))

        # A correct implementation should give a result around -0.481
        print(sin_MSE_dt2(theta, x, y))

```

```

2.0556172417592418
-0.4808513527956513

```

Let's now implement gradient descent.

```

In [ ]: def init_theta():
        """Creates an initial theta [0, 0] of shape (2,) as a starting point for
        return np.array([0, 0])

    def grad_desc(loss_f, gradient_loss_f, theta, data, num_iter=20, alpha=0.1):
        """
        Run gradient descent update for a finite number of iterations and static

        Keyword arguments:
        loss_f -- the loss function to be minimized (used for computing loss_hist)
        gradient_loss_f -- the gradient of the loss function to be minimized
        theta -- the vector of values theta to use at first iteration
        data -- the data used in the model
        num_iter -- the max number of iterations

```

```

alpha -- the learning rate (also called the step size)

Return:
theta -- the optimal value of theta after num_iter of gradient descent
theta_history -- the series of theta values over each iteration of gradien
loss_history -- the series of loss values over each iteration of gradien
"""

theta_history = []
loss_history = []
for i in range(num_iter):
    theta_history.append(theta)
    loss_history.append(loss_f(theta, data['x'], data['y']))
    d_b = gradient_loss_f(theta, data['x'], data['y'])
    theta = theta - alpha * d_b

return theta, theta_history, loss_history

```

```

In [ ]: # Initialize the parameters theta
        theta_start = init_theta()

        # Run gradient descent to update theta
        theta_hat, thetas_used, losses_calculated = grad_desc(
            sin_MSE, sin_MSE_gradient, theta_start, df, num_iter=20, alpha=0.01
        )

        # Print the theta and loss in each iteration
        for b, l in zip(thetas_used, losses_calculated):
            print(f"theta: {b}, Loss: {l}")

```

```

theta: [0 0], Loss: 20.859191416422235
theta: [0.26010575 0.26010575], Loss: 10.34954705433671
theta: [0.43992234 0.35498395], Loss: 6.888324886941001
theta: [0.58352797 0.38144639], Loss: 4.958929972250028
theta: [0.70229893 0.39049608], Loss: 3.6621739866792846
theta: [0.80098571 0.39364501], Loss: 2.769592703652369
theta: [0.88307172 0.39431768], Loss: 2.152339203791021
theta: [0.9513768 0.3937597], Loss: 1.7248463835940306
theta: [1.00822852 0.39249503], Loss: 1.4285147701487082
theta: [1.05555802 0.39077126], Loss: 1.222918680676549
theta: [1.09497047 0.38871812], Loss: 1.0801096115485163
theta: [1.12780113 0.38641079], Loss: 0.980751181007712
theta: [1.15516112 0.38389762], Loss: 0.911462307351967
theta: [1.17797534 0.3812131 ], Loss: 0.8629830069079113
theta: [1.19701372 0.37838426], Loss: 0.8289059822784286
theta: [1.21291726 0.37543391], Loss: 0.8047988117285686
theta: [1.2262196 0.37238234], Loss: 0.7875962204766015
theta: [1.23736498 0.36924812], Loss: 0.7751792441769542
theta: [1.24672317 0.36604856], Loss: 0.7660837758507504
theta: [1.25460183 0.36279985], Loss: 0.759298710557517

```

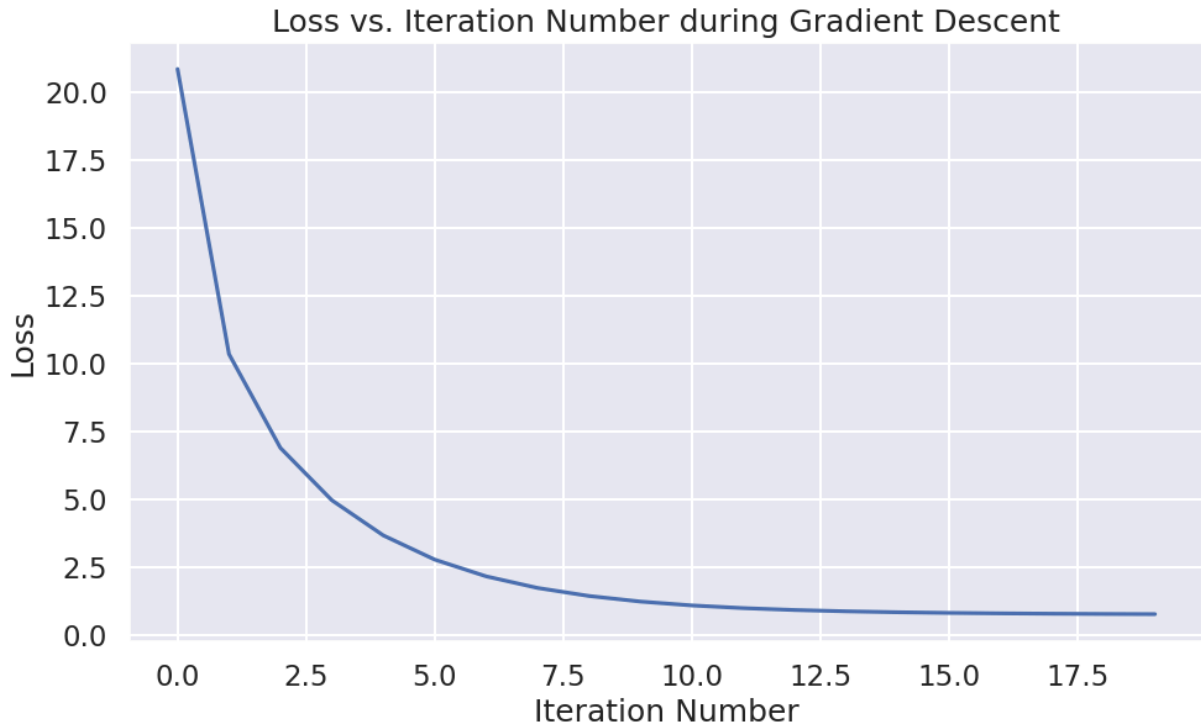
Insert code below to create a 2D plot, where the x axis is the iteration number ,and the y axis is the loss in each iteration

```

In [ ]: plt.figure(figsize=(10, 6))
        plt.plot(range(len(losses_calculated)), losses_calculated)

```

```
plt.xlabel("Iteration Number")
plt.ylabel("Loss")
plt.title("Loss vs. Iteration Number during Gradient Descent")
plt.grid(True)
plt.show()
```



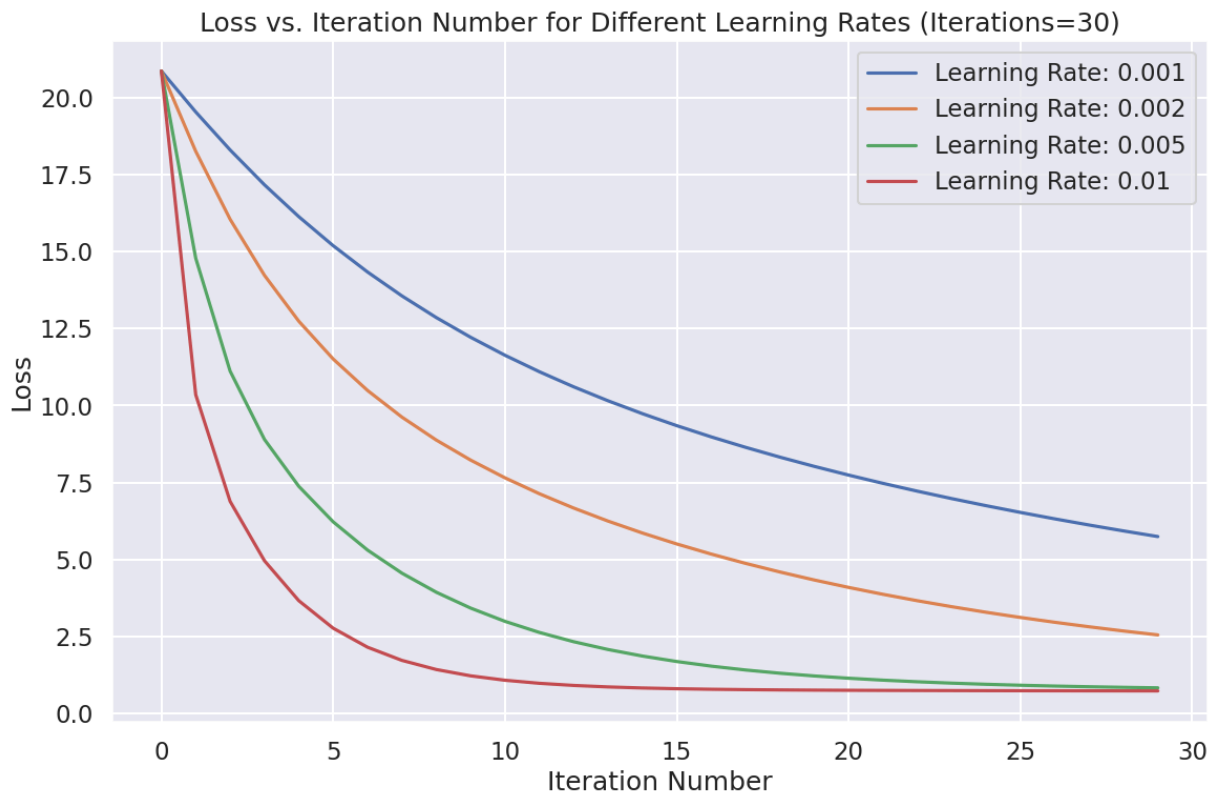
Insert code below, try each of the learning rate of [0.001, 0.002, 0.005, 0.01], initialize the parameters theta, run the gradient descent to update theta with the specific learning rate and max iteration of 30, and show the plot loss v.s. iteration.

```
In [37]: learning_rates = [0.001, 0.002, 0.005, 0.01]
num_iterations = 30

plt.figure(figsize=(12, 8))

for lr in learning_rates:
    theta_start = init_theta()
    _, _, losses_calculated = grad_desc(
        sin_MSE, sin_MSE_gradient, theta_start, df, num_iter=num_iterations,
    )
    plt.plot(range(len(losses_calculated)), losses_calculated, label=f'Learn

plt.xlabel("Iteration Number")
plt.ylabel("Loss")
plt.title(f"Loss vs. Iteration Number for Different Learning Rates (Iteratio
plt.legend()
plt.grid(True)
plt.show()
```



Based on the plot, give your observation of the effect of learning rate

Is it always better to use a larger learning rate? why? (Hint: Repeat the same plot above with learning rate 0.5 and observe the results before answering this question.)

ANSWER: It is not always better to use a larger learning rate because it can cause the model to overshoot, oscillate, or diverge. It sometimes provides better initial performance, but is detrimental overall.

Let's visually inspect our results of running gradient descent to optimize θ .

The code below plots our x -values with our model's predicted $\hat{y}$ -values over the original scatter plot. You should notice that gradient descent successfully optimized θ .

```
In [38]: theta_init = init_theta()

theta_est, thetas, loss = grad_desc(sin_MSE, sin_MSE_gradient, theta_init, d
```


Plotting our model output over our observations shows that gradient descent did a great job finding both the overall increase (slope) of the data, as well as the oscillation frequency.

```
In [39]: x, y = df['x'], df['y']
y_pred = sin_model(x, theta_est)
plt.plot(x, y_pred, label='Model ( $\hat{y}$ )')
plt.scatter(x, y, alpha=0.5, label='Observation ( $y$ )', color='gold')
plt.legend();
```

<>:3: SyntaxWarning:

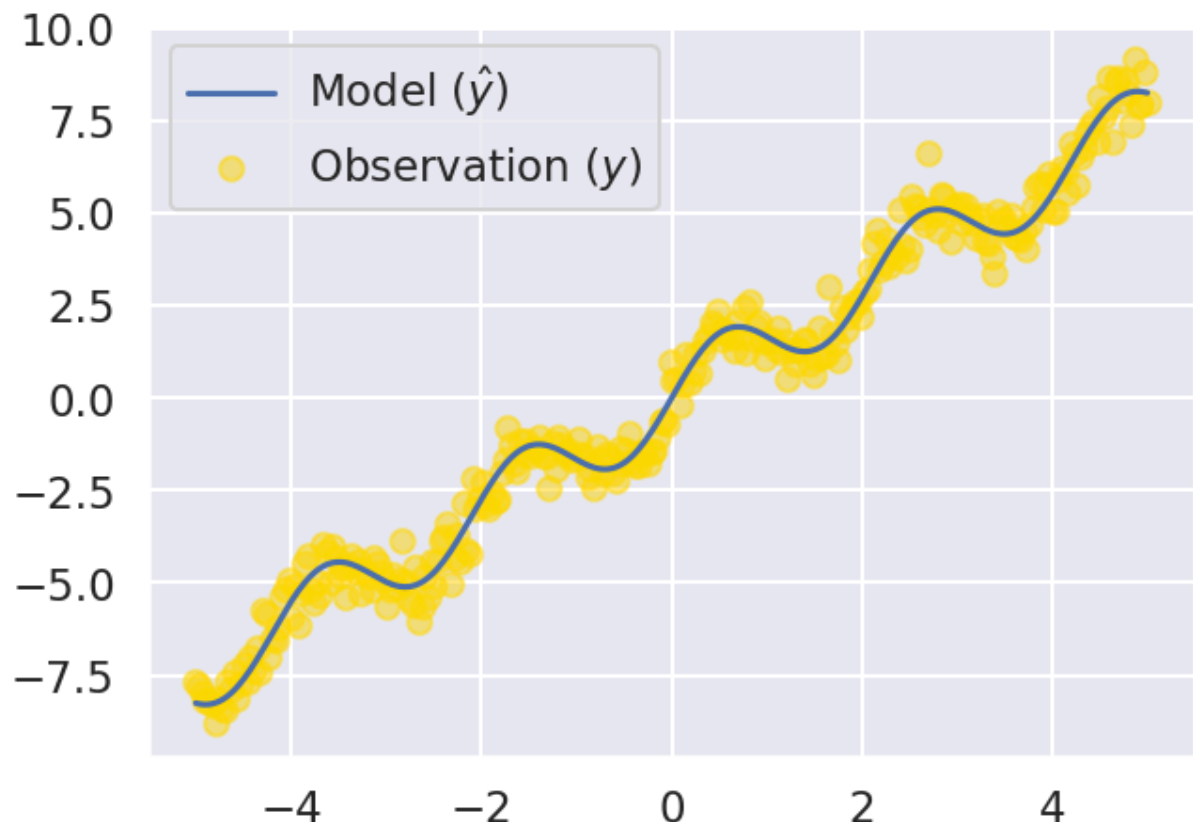
invalid escape sequence '\h'

<>:3: SyntaxWarning:

invalid escape sequence '\h'

/tmp/ipython-input-3844925747.py:3: SyntaxWarning:

invalid escape sequence '\h'



Lab 8 is now complete. Make sure all cells are visible and have been run (rerun if necessary).

The code below converts the ipynb file to PDF, and saves it to where this .ipynb file is.

```
In [ ]: NOTEBOOK_PATH = "/content/drive/MyDrive/ECEN250/Lab8/ECEN250-F25-Lab8.ipynb"
! pip install playwright
! jupyter nbconvert --to webpdf --allow-chromium-download "$NOTEBOOK_PATH"
```

Download your notebook as an .ipynb file, then upload it along with the PDF file (saved in the same Google Drive folder as this notebook) to Canvas for Lab 8. Make sure that the PDF file matches your .ipynb file.